

Memoria virtual

La evolución tecnológica del hardware permite que las CPUs soporten grandes espacios de direcciones. Una cpu moderna puede soportar espacios direcciones de 64 bits (128 terabytes) o más.

Esto permite el desarrollo de programas muy grandes. Comúnmente en un sistema moderno la cantidad de memoria RAM requerida por los procesos es mayor a la disponible físicamente por el hardware.

Un proceso no tiene por qué estar cargado en memoria completamente, sino sólo las partes en uso en un momento determinado.

Con paginado es posible cargar (*swap-in*) o desalojar (*swap-out*) partes de un proceso con una granularidad del tamaño de páginas.

El *swap-in* se requiere cuando un proceso referencia a una página que reside en disco.

El *swap-out* se requiere cuando el sistema requiere liberar una página en RAM para usarla ante la no existencia de páginas libres. Esta operación, comúnmente es invocada por el *memory allocator*.

Este proceso se conoce como *swapping* o *demand paging*.

Para cada página se debe mantener su estado, es decir si está mapeada a una página física en RAM o en un bloque de disco. Esto comúnmente se hace almacenando la *physical page number (ppn)* o un *disk block number (dbn)* y el bit *valid* (o *present* en algunas arquitecturas) en 1 o cero, respectivamente.

Con paginado es posible *extender* la memoria RAM en dispositivos de almacenamiento con bloques del mismo tamaño como se muestra en la siguiente figura.

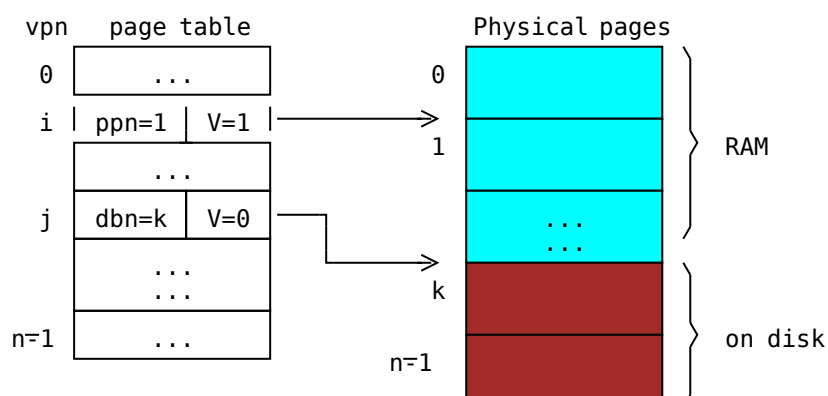


Figura 1: Memoria virtual.

En la figura anterior se muestra un esquema con un espacio de direcciones hasta n páginas con un hardware con una memoria principal física de k páginas de capacidad. Las páginas en disco se almacenan en un área conocida como *swap area*.

La página virtual j reside en disco, ya que la *pte j* tiene apagado el bit V (*valid*) pero contiene un *disk block number*.

Cuando una instrucción intente acceder a la página j se producirá una excepción. El kernel deberá hacer un *swap-in*:

1. Reservar una página en memoria (sea $ppn=X$)
2. Leer el contenido del bloque k en disco en la página X
3. Mapear la *pte j* a X : Hacer $ppn=X$ y bit $V=1$

El *swap area* puede ser un archivo o una partición con el formato adecuado. Por ejemplo, GNU-Linux permite configurar el *swapping area* en un archivo o en una partición con formato del tipo *swap*. El uso de un sistema de archivos especial en lugar de un archivo mejora sustancialmente el rendimiento del sistema.

Mapeo de archivos en memoria

Una llamada al sistema del tipo `va = mmap(fd, mode)` crea un *mapping* de direcciones lógicas o virtuales a los contenidos del archivo en disco y retorna la dirección lógica del comienzo (base) del contenido mapeado en la memoria.

El contenido del archivo se divide lógicamente en bloques del tamaño de una página. Mmap inicialmente no asigna memoria ni carga el contenido del archivo, sólo crea el espacio de direcciones (setea entradas en la tabla de páginas, indicando que están en los correspondientes bloques del disco).

La carga de los datos del archivo se harán en forma transparente por el OS bajo demanda, a medida que el proceso acceda a direcciones que pertenecen al espacio de memoria creado.

Esto permite el acceso a los datos del archivo accediendo directamente a memoria sin usar llamadas al sistema `read` o `write`.

En la primer referencia de la forma `va[i]`, donde `i` es un *offset* desde el comienzo del archivo, el sistema asignará una página de memoria y cargará en ella los contenidos de bloque correspondiente a esa página.

El siguiente programa muestra un ejemplo de su uso en un SO hipotético.

```

// let myfile.dat a file with contents with more than 4096 bytes as belc
// After mmap(), we can see it as a character array buf[] as
//
// buf --> +-----+ 0
//         |Hello world\0                               |
//         |...                                           | 4095
//         +-----+ 4096
//         |Second page\0                               | ...
//         |padding (zeroes) by mmap                     |
//         +-----+ 8192
...
int fd = open("myfile.dat", O_RDONLY);
char *buf = mmap(fd);
printf("%s\n", buf);
// output: Hello world
printf("%s\n", buf + 4096);
// output: Second page
printf("%s\n", buf + 7005);
// output: empty string (zeroes in second page)
printf("%s\n", buf + 8192);
// page fault (maybe)
...

```



Listado 1: Ejemplo de uso de mmap.

Una llamada al sistema del tipo `unmmap(va)` libera la memoria usada para los bloques del archivo y mapping.

El *mapping* consiste en agregar *pte's* que referencien a números de bloques de disco en lugar a números de páginas de memoria. En algunos sistemas tipo UNIX el mapping se hace a partir de la dirección *brk* del proceso, la cual indica el comienzo de su bloque de espacios de direcciones libres.

Comúnmente se setean flags de permisos de acceso pero con el bit *V* en cero como se muestra en la siguiente figura. Las páginas en sí en RAM para los datos del archivo no se reservan (`alloc`) inicialmente.

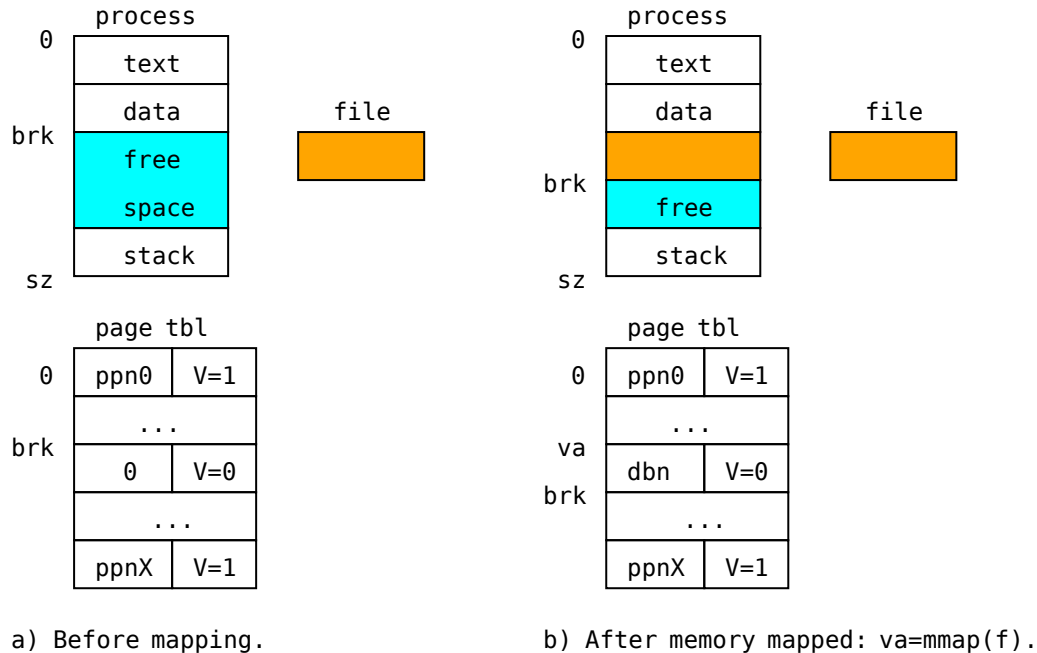


Figura 2: Ejemplo de mapeo de un archivo (de tamaño no mayor a una página) en memoria.

En la figura 2, `ppnX` refiere a una *physical page number X* en memoria. El valor `dbn` refiere al número de bloque (de igual tamaño que una página) del disco. La llamada al sistema `mmap` retorna `va` y avanza el puntero `brk` del proceso.

Esta configuración de la *page table* hace que cuando el programa acceda a la dirección que hace referencia a una página mapeada al archivo y ésta no está en la memoria, se produzca una excepción ya que la *pte* referenciada tendrá `V=0`.

El kernel podrá verificar si corresponde a una dirección lógica válida (el campo `ppn` no es cero) se deberá a que corresponde a una página que reside en disco y así podrá procesar la excepción:

1. Reservar (*alloc*) una página física. Sea si $ppn = \alpha$.
2. Cargar el bloque de disco `dbn` en la página obtenida.
3. Mapearla: setear $pte.ppn = \alpha$ y los flags (`V=1`) en la *pte* correspondiente de la tabla de páginas.

Al retornar de la excepción la cpu podrá ejecutar exitosamente la instrucción que la causó y accederá al contenido de esa porción del archivo.

En el caso que el proceso modifique el contenido (copia) en memoria del archivo, al liberar el mapping, el SO puede *escribir los datos* en el archivo. Notar que el bit *dirty* indica qué página fue modificada.

Carga de programas bajo demanda

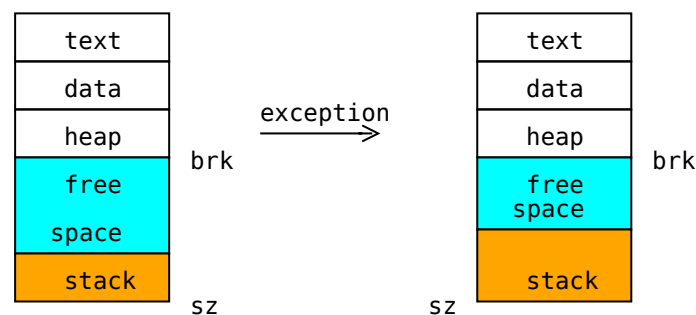
La llamada al sistema `exec(program-file, args)` debe cargar los contenidos del ejecutable en disco a memoria. Si el programa es grande, ésto puede requerir tiempo y memoria.

Si `exec` mapea los segmentos cargables del ejecutable en memoria con `mmap` y eventualmente cargando sólo la página de código que contiene el *entry point* es posible realizar la carga del programa bajo demanda, a medida que la ejecución del proceso progrese y vaya referenciando direcciones de instrucciones y datos.

Stack con crecimiento dinámico

El problema a resolver en la gestión de la memoria es determinar cuánta memoria se debería asignar para el stack.

Con memoria virtual, podría mapearse inicialmente sólo una página en una dirección lógica alta y el SO podría asignar y mapear más páginas bajo demanda, es decir a medida que el proceso intenta escribir sobre el tope del espacio del stack asignado que generará una excepción.



De esta manera el stack *crece* bajo demanda como se muestra en la figura de arriba. El *heap* generalmente se expande hacia abajo asignándole mas espacio usando la llamada al sistema `sbrk(size)` .

Memoria compartida

Para que dos o mas procesos o el kernel y los procesos puedan comunicarse usando la memoria se requiere que uno o más bloques de memoria físicos pertenezcan a sus espacios de direcciones. Esto se logra *mapeando* en cada proceso un espacio de direcciones lógicas que resuelvan a las mismas direcciones físicas.

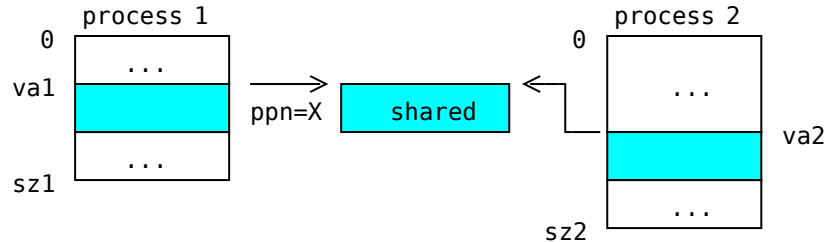


Figura 2: Esquema de dos procesos compartiendo una página.

En la figura 2 dos procesos comparten una página. Las direcciones lógicas *va1* y *va2* comúnmente son diferentes pero ambas están mapeadas al mismo número de página física (*ppn X*).

Los SO tipo UNIX, proveen llamadas al sistema para que los procesos puedan compartir memoria.

- `int shmget(key, size, flags)` : Crea u obtiene in descriptor para un bloque de memoria compartida con identificador *key* de tamaño *size*.
- `void *shmat(m)` : Obtiene la dirección lógica del descriptor *m* retornado por `shmget` .
- `int shmdt(address)` : Libera (*detach*) el mapping.

Obviamente, los procesos que se comuniquen mediante memoria compartida deberán usar algún mecanismo de sincronización (como semáforos) para evitar condiciones de carrera en el acceso a los datos o recursos compartidos.

Esta técnica junto con el mapeo de archivos en memoria se usa para *mapear* una ***biblioteca compartida*** en el espacio de direcciones de un proceso.

Esto complica la liberación de memoria de un proceso, por ejemplo en un `exit()` ya que hay que liberar sólo páginas no compartidas.

Una solución a esto es asociar a cada *shared memory region* un contador de *sharing* (o referencias), indicando el número de procesos que la comparten. Este contador se incrementa en un `va=shmat(m)` y se decrementa en cada `shmdt(va)` . Cuando llegue a cero, es seguro liberar la memoria compartida.

Copiar al escribir (COW)

La llamada al sistema `fork()` debe copiar la imagen del proceso padre en el hijo. Esta duplicación puede requerir altas demandas de memoria y copias de grandes cantidades de bloques.

En lugar de hacer la copia completa se copia la tabla de páginas, haciendo que ambos procesos compartan la memoria. Para lograr el efecto de *confinamiento* de cada proceso, en ambas tablas de páginas se quitan los permisos de escritura en cada *pte*, quedando la memoria compartida de sólo lectura. A cada proceso debería asociarse una *shared memory block* para todo su espacio de direcciones.

Cuando uno de los dos procesos desea escribir en una página (sea la *virtual page number ppn*) se producirá una *excepción* ya que no tiene permiso de acceso para escritura. El kernel captura la excepción, reconoce que se produce por la falta de permisos pero es una dirección válida (y *shared*). Entonces hace los siguientes pasos:

1. Asigna (alloc) una nueva página. Sea una con $ppn=n$.
2. Copia el contenido de la página original en la nueva.
3. Mapea la nueva página: $ppe.ppn=n$ y $ppe.W=1$ en la *ppe* correspondiente del proceso corriente.
4. Hace $W=1$ en la *pte* correspondiente del proceso padre o hijo, según corresponda.

Como en cada excepción recuperable, se debe preparar la cpu para que re-intente la ejecución de la instrucción que causó la falta en el proceso.

Este mecanismo se conoce como *copy on write (COW)*.

Para generalizar COW se necesita llevar la pista de las páginas compartidas entre procesos, comúnmente usando *shared memory regions* en los descriptores de tareas.

Algoritmos de reemplazo de páginas

Cuando el sistema requiere asignar memoria (ej: `allocpage()`) ya sea para un proceso o para el kernel, es posible que no exista ninguna página libre. Esta situación se conoce como *page fault*.

Con swapping, el sistema podrá elegir una página (*víctima*) en uso, desalojarla a disco (*swap-out*) y asignarla para satisfacer el requerimiento.

Uno de los problemas a resolver es determinar cómo elegir la víctima. Lo ideal sería que ésta fuese una página que será referenciada más en el futuro (eventualmente nunca). Esto obviamente no es computable ya que requiere predecir el futuro.

Ya se ha discutido en estas notas que una forma de intentar predecir el comportamiento futuro es hacerlo en base al comportamiento pasado. Los siguientes algoritmos intentan seleccionar la víctima mas adecuada en base a diferentes heurísticas.

FIFO

El sistema mantiene una cola de las páginas en uso con la última asignada como último elemento. En un *page fault* El algoritmo FIFO selecciona la página a la cabeza y se encola. Esta página es la que se ha desalojado hace más tiempo.

Second chance

El principal problema del algoritmo FIFO es que puede elegir una página usada muy frecuentemente.

Este algoritmo se basa en FIFO con una simple modificación.

En lugar de asignar la página de la cabeza, se inspecciona si fue accedida (bit $A=1$). Si es así, se hace $A=0$ y se encola la página (se mueve al final). Este proceso continúa hasta encontrar una página a la cabeza con $A=0$.

Este algoritmo selecciona una página que no ha sido víctima hace tiempo y no fue accedida desde el último período o *page fault*.

Una variante de este algoritmo es no usar una cola, sino una lista circular de las páginas usadas con un índice o referencia: el *hand* (la mano) que apunta a la página mas vieja (no seleccionada como víctima).

En un *page fault*, se verifica el valor del bit A . Si es cero, se elige como víctima, sino se hace $A=0$ y se avanza el *hand* repitiendo el proceso.

Esta implementación se conoce como el algoritmo del reloj *clock*.

Menos recientemente usada (LRU)

Este algoritmo tiene como objetivo elegir como víctima una página que no ha sido usada en un lapso de tiempo determinado. La implementación de este algoritmo no es fácil de realizar eficientemente sin el soporte adecuado de hardware.

Esto requiere mantener una lista de todas las páginas ordenadas por tiempo de acceso (de menor a mayor). Los tiempos de acceso pueden ser *ticks* (o múltiplos). En cada intervalo de tiempo se mueven las páginas accedidas al final de la lista.

Una implementación alternativa de LRU se conoce como *not frequently used (NFU)*.

NFU asocia un contador a cada página, inicialmente en cero. Periódicamente, el SO analiza cada *pte* de cada tabla de páginas y suma el bit A al contador. Se mantiene una lista de páginas ordenada por el contador (de menor a mayor).

Ante un *page fault*, la *víctima* es la página en la cabeza de la lista.

Los siguientes problemas son difíciles de resolver eficientemente:

1. Se deben recorrer todas las tablas de páginas periódicamente.
2. Siempre el contador se incrementa, es decir *nunca olvida*.

El punto 1 puede resolverse haciéndolo en momentos de ocio del sistema y en forma incremental.

El punto 2 puede resolverse haciendo un *shift* del contador a derecha de un bit (dividir por dos) antes de sumarle el bit *A*. Esto implementa una idea de *aging* y trata de prevenir que páginas que se han tenido muchos accesos en el pasado pero que actualmente no se acceden más o muy poco, vayan adelantándose en la lista.

El kernel de Linux, actualmente usa una versión de *software LRU (NFU)* de dos niveles. Mantiene dos listas de páginas: *activas* (accedidas recientemente) e *inactivas*. Las páginas inactivas son candidatas a ser reemplazadas.

No recientemente usada (NRU)

Este algoritmo selecciona una página a reemplazar en base al soporte del hardware descrito. El objetivo es determinar qué páginas han sido accedidas en el último período de tiempo.

Periódicamente, cada n ticks por ejemplo, el bit *A* (*accessed*) se pone en cero (*cleared*).

Ante un *page fault*, para elegir una víctima las páginas se clasifican en base a:

1. $A=0, D=0$: No accedida en el último período.
2. $A=0, D=1$: Escrita en el período anterior ($A=0$ es producto del *clearing* periódico).
3. $A=1, D=0$: Leída en el último período.
4. $A=1, D=1$: Accedida en el último período, escrita en algún momento.

Se elige una víctima en la menor clase no vacía.

Este algoritmo se conoce como *software LRU* ya que se puede implementar en arquitecturas modernas sin soporte para LRU.

Working set

La idea de este algoritmo se basa en que los procesos en un período de tiempo τ acceden a un conjunto de páginas. Ese conjunto de páginas recientemente accedidas conforman un *working set* del proceso.

Es fácil de ver que un proceso en una iteración o si está accediendo a datos almacenados en forma contigua, referenciará muchas veces un conjunto relacionado de páginas.

El objetivo es registrar para cada página si ésta fue accedida en el intervalo de tiempo τ . Cada página se asocia a un reloj lógico (*ticks*) con el tiempo del último acceso (*tick*).

Ante un *page fault* se recorre la tabla de páginas inspeccionando el bit *A*. Si $A=1$, se actualiza el valor del reloj asociado a la página y se agrega al *working set* del período corriente.

Si $A=0$ se deberá determinar si puede elegirse como víctima. Si el tiempo de acceso está dentro del intervalo actual ($tick - last_access_{page} < \tau$) se mantiene en el *working set*. Si no se considera fuera del *working set*. Se elige como víctima una página fuera del *working set* con menor valor de reloj.

Para evitar recorrer toda la tabla de páginas puede usarse una mejora basada en el *algoritmo de reloj* descripto arriba.

El algoritmo usando en Linux de alguna manera intenta que la lista de páginas *activas* representen el *working set* actual del sistema.

Otras consideraciones

Los algoritmos descriptos no consideran otros aspectos de selección de páginas a reemplazar como por ejemplo, su importancia en el sistema.

Una consideración a toma en cuenta es si el algoritmo de reemplazo de páginas debe ser global (sin tener en cuenta por quién está usada la página) o local (se consideran las páginas del proceso corriente, por ejemplo).

Por ejemplo, no es lo mismo reemplazar una página usada por el kernel o una biblioteca compartida que una usada por un proceso.

Para evitar esto, comúnmente ciertas páginas se *protegen* o marcan de ser elegidas como víctimas. Otro ejemplo son las que pertenecen a espacios de direcciones de dispositivos mapeados en memoria (*MMIO*).

Uno de los peligros del *page swapping* es la *sobre paginación (trashing)*, la cual es un estado en el que el sistema está ejecutando *out of memory* y ocurren repeditamente los siguientes eventos:

1. Se debe hacer un *swap-out* para liberar memoria.
2. La página debe volverse a cargar porque es referenciada.

Estos pasos se repiten a una alta frecuencia. El sistema comienza a degradar el rendimiento de manera considerable y el progreso de los procesos se hace muy lento.

Caso de estudio: GNU-Linux

Linux implementa un algoritmo *LRU* aunque la lista de páginas no repleja exactamente un orden por tiempo de acceso.

Mantiene dos listas: *active_list* e *inactive_list*. El objetivo es mantener en *active_list* un *working set* de todos los procesos e *inactive_list* la lista de candidatos a reemplazar.

Estas listas se actualizan periódicamente. Esto hace que la política sea global.

[< Anterior](#)

Gestión de la memoria

[Próximo >](#)

Entrada/salida